Solver-First Itinerary Planning for ChinaTravel: Where the Language Model Helped, and Where We Measured That It Did Not

Stefan Schneider[1] , **Fabian Missbrenner**[2]
[1]Digital Rail (DB InfraGO AG), [2]TU Berlin
stefan.sh.schneider@deutschebahn.com, mail@fabimi.net

Abstract

We describe team `fabiundstefan`'s Phase 2 entry to the Travel Planning Challenge at IJCAI 2026. The system is a deterministic, solver-first planner: it extracts branch-local constraints from the visible query text, builds itineraries whose every fact is read from the sandbox environment, and emits a plan only after the official schema, commonsense and logical checks accept it. The submitted path makes **zero language-model calls and zero network calls**, and scores **57.99 overall** (final pass rate 33.0) on the released `familiar_100` split while using 26% of its five-hour budget. That the entry to a *language agent* challenge contains no language model at inference time is a measured decision, not an omission: we built LLM-backed constraint extraction and LLM-backed plan repair, and both were screened out — the parse audit costs -3.2 score once it actually fires, and the refiner committed 0 of 18 proposed edits. Meanwhile the model was central to *building* the system: 172 of 759 commits are agent-co-authored. We argue this split — LLM in the outer development loop, solver in the inner inference loop — is the useful finding, and we report the measurement discipline that made it visible, including a failure mode in which an LLM component looks beneficial precisely because it silently never ran.

1 Introduction

ChinaTravel [Shao *et al.*, 2024] asks an agent to turn an open-ended natural-language request ("three days in Suzhou, we must be at the Humble Administrator's Garden between 9 and 11, budget under 3000 yuan") into a multi-day itinerary whose attractions, restaurants, hotels, inner-city legs and intercity transport all exist in a sandbox database, and whose every stated requirement is checked by a compositional verifier. Unlike slot-filling benchmarks [Xie *et al.*, 2024], requirements are compositional, sometimes disjunctive, and often implicit. The benchmark's own study reports a large gap between purely neural agents and neuro-symbolic ones, the latter reaching roughly 37% constraint satisfaction on human queries — about an order of magnitude better.

Our Phase 2 system takes that gap to its conclusion. It is a deterministic planner with an acceptance gate: nothing is emitted that the official checkers have not already accepted, and no itinerary fact is generated as text. Phase 1 (offline prediction on released data, where oracle constraint programs may be used locally) validated the approach — overall 98.98 on our local run of the evaluator of the time, second on the public leaderboard — and Phase 2 removes that privilege: oracle fields are stripped before the agent sees a query.

The interesting part is what happened to the language model. The organizers even provide one on the evaluation host (a local SGLang deployment of `Qwen3.6-27B`), so using it would have been free. We nonetheless ship a system that never calls it. This is the outcome of a deliberate experiment rather than a prior: we implemented the two places where an LLM plausibly helps a solver-first pipeline — correcting the constraint extraction, and proposing repairs to a plan the checker rejected — measured both, and turned both off.

Contributions. (i) A deterministic, interruption-safe planner and whole-run scheduler that fits an unbounded search problem into a fixed five-hour wall clock (Section 2). (ii) A diagnosis that our score is *representation*-bound, not compute-bound: five times the budget buys $+0.64$ overall, while one missing solver move buys $+10$ final pass rate (Section 3). (iii) Ablations for every promoted mechanism and for both LLM stages (Section 4). (iv) Methodology: how the model was used to build the system, and two traps that make LLM-component measurements lie (Section 5).

2 System

Execution contract. The agent path receives the visible query text and the environment tools, nothing else. Oracle verifier fields are removed before parsing. Every POI, price, opening hour, schedule and travel time comes from a `WorldEnv` query. Installation is eight PyPI packages; there is no model download, no external index and no egress. The whole run is one command with no flags, as the organizers specified.

Per-query pipeline. *Parse* extracts constraints from the query text with a rule-based grammar over the benchmark's constraint families. *Normalize* turns them into solver-facing variants, resolving named entities against the database,

canonicalizing concept labels, and — importantly — keeping disjunctive requirements as separate branches rather than flattening them into one conjunction, end to end through solving and checking. *Solve* constructs a base itinerary greedily, then runs bounded randomized restarts over a small parameter space (day offsets, hotel choice, attractions per day, visit durations, forced transport mode, meal offsets), with an optional CP-SAT tier [Perron and Didier, 2024] for candidate pools and hard residuals, and acceptance-gated repair and densification passes. *Finalize* selects a candidate under an explicit acceptance ordering — a fully accepted plan wins; failing that, the best schema- and commonsense-valid candidate is preferred over a higher-scoring invalid one, because the scored logical pass rate is conditional on the commonsense pass — and emits telemetry. This is the LLM-Modulo arrangement [Kambhampati *et al.*, 2024] with the generator replaced by a deterministic one.

Whole-run scheduling. Phase 2 grants one five-hour envelope for 100 queries, and every query is weighted equally, so a structurally unsolvable query must not starve the rest of the split. Rather than giving each query $1/n$ of the clock, we run passes across $cores - 1$ worker processes (a single solve is single-threaded): **triage** solves every query once at a small budget, so the split has real plans early; **repair** revisits only constraint-failing queries at a growing budget, most-nearly-passing first; **polish** improves preference terms on already-passing queries with whatever time is left — last, because the evaluator zeroes all preference terms of a failing query. Three properties make the run safe: a schema-valid plan for every query is written *before* the first solve; every later result is scored oracle-free and kept only if it beats what is on disk (keep-best, so no pass can regress a query); and the scheduler refuses to start work the envelope cannot finish. The run can be killed at any instant and still leaves a complete, non-regressed set of plans.

3 Where the points are

The official score is $0.10\,\mathrm{MicEPR} + 0.10\,\mathrm{MacEPR} + 0.25\,\text{C-LPR} + 0.40\,\mathrm{FPR} + 0.05(\mathrm{DAV} + \mathrm{ATT} + \mathrm{DDR})$, where FPR is the fraction of queries satisfying *every* constraint and the three preference terms (attraction diversity, inner-city transport, restaurant diversity) are zeroed on any query that does not. At our operating point one FPR point is worth about 0.85 overall, whereas perfecting all three preference terms is worth about 1.5 in total. FPR is the only number that matters.

The search is bimodal. Per-query telemetry from a one-hour diagnostic run shows two populations and no middle. Of 42 triaged queries, the 15 that were accepted were accepted by the *first constructed plan*, in a median of 3.2 seconds with a median of 0 restarts. The 27 that failed spent their entire slice (median 22.7 s of 23 s) on a median of 39 randomized restarts and found nothing acceptable. Giving the failures $2.5\times$ the time raised the median restart count to 166 and flipped **one query in 68**.

So more compute is not the answer, and we measured that rather than assuming it. Four whole-split runs at different envelopes (Table 1, lower block) score $57.35/57.81/57.86/57.99$ at $1/2/3/5$ hours. Every knob we have — budget, restart cap, worker count, fresh seeds — is the same intervention: draw more tuples from a parameter space of a few thousand combinations. The measurement says that space does not contain the fix.

One constraint family accounts for almost all of the loss. Feeding the solver oracle constraints and attributing its failures by family, the shape "be at *this named POI* within *this time window*" was the *only* failing family in 45 of 61 failing queries. The sampler has no parameter that pins one activity inside a window and reflows the rest of the day around it, which is exactly why extra draws never help. Adding that one move (`TPC_WINDOW_ANCHOR`: a window-anchored day relayout) at identical budget, restart count and seed moved FPR $32.5 \rightarrow 42.5$ on both the solver-only and the end-to-end axis (Table 2). The gap was representation, not search time.

Parsing is not the bottleneck either. Rule-based extraction reaches $F_1 = 0.959$ against the oracle constraints with no missing family, and — the stronger statement — handing the solver the *oracle* constraints instead of parsed ones does not raise FPR (23.75 solver-only vs. 25.00 end-to-end on the same per-query contract). A perfect parser is worth approximately nothing here. This is the quantitative reason the LLM parse stage had little to win, and Section 4.1 reports what it actually did.

4 Experiments

Setup. All numbers are on the released `familiar_100` data with the upstream evaluator at `c89cfca` (our evaluation code reproduces upstream exactly; the only local deltas are lazy tool loading and defensive guards on malformed plans, never a scoring difference). Whole-split *rows* use the real 5 h envelope with 3 workers. Paired *screens* use an 80- or 30-query subset at a 90 s per-query budget with a fixed seed; they are a triage instrument for deciding what deserves a full run, their absolute values are not comparable to rows, and they never promote a default on their own.

4.1 The language-model ablation

We measured the LLM in both places it could act, with the same paired screen used for every deterministic flag, on a shared all-LLM-off baseline, and with per-query telemetry asserting that the model actually answered.

Constraint extraction (parse audit). The model receives the query, the rule-based extraction and the constraint schema, and proposes patches; every proposal round-trips through the deterministic parser and a whitelist, so it can never introduce an unrepresentable constraint. With the audit firing on 30 of 30 queries, FPR was unchanged and the continuous score fell $47.38 \rightarrow 44.18$ (-3.20): the model rewrites preference-side detail it cannot improve on. That is consistent with Section 3 — even a *perfect* parse is worth nothing — so the stage was never going to pay for itself.

Plan repair (bounded refine). When the checkers reject a plan, the model is shown the plan and the failing clauses and proposes edits; each edit must ground against `WorldEnv` and re-pass the full checker before it is committed. It was

Metric	value	weight	contributes
FPR	33.00	0.40	13.20
C-LPR	87.83	0.25	21.96
MicEPR	99.48	0.10	9.95
MacEPR	97.00	0.10	9.70
DAV	18.15	0.05	0.91
ATT	26.25	0.05	1.31
DDR	19.39	0.05	0.97
Overall			**57.99**

Envelope	1 h	2 h	3 h	5 h
Overall	57.35	57.81	57.86	57.99
FPR	31.0	32.0	32.0	33.0

Table 1: Top: the submitted configuration on the whole 100-query split under the real five-hour envelope, 0 empty plans, finishing in 1.31 h (26% of the budget). Bottom: the same system at four envelopes — five times the time buys $+0.64$ overall, so the system is not compute-bound. (Rows `20260804T201638Z`, `213636Z`, `231910Z`, `221103Z`.)

called on 18 rejected plans and committed **0** edits: 8 proposals referenced things that do not exist in the database, 5 produced plans the checker still rejected, 5 errored. Meanwhile its reserved time slice raised timeouts from 11 to 19. The telemetry is more informative than the score here: the failure is not that the model's ideas are bad, it is that they are not *groundable*, and grounding is the whole difficulty of this benchmark.

Why we did not simply keep them off by default and on when available. Every LLM second inside the shared five-hour envelope is a second not spent in search, and an external dependency is a reliability liability in a harness where a crashed worker still exits 0 while shipping empty plans. Given a measured cost and no measured benefit, the defensible default is off — and "off" is enforced: with no backend configured the pipeline is byte-identical to the deterministic floor, asserted by unit test.

What this does not show. These are screens at reduced budget, not full rows, and they cover two *specific* uses. The one untried use is the LLM as a *move proposer* — not editing a finished plan, but suggesting which structural move (shift, shorten, relocate across days, insert a missing activity) a failing query needs, with the deterministic solver executing and the checker gating it. Section 3 says such moves are exactly where the remaining points are, so this is the one direction we would still expect to pay, and we keep it open until the submission deadline.

4.2 Error analysis

After the window-anchored relayout, 43 of 80 validation queries still fail a timed-visit clause, 33 of them with *nothing else* failing — the same family, one clause short. Classifying those 57 failing clauses by the move they need: forward-shift 21, absent POI 15, shorten 11, backward-shift 10. Twelve sole-blocker queries need only a forward shift or a shortening (11 of them are off by 5–19 minutes), 6 need only the insertion of a POI that is not in the plan at all, and 8 need a cross-day relocation. The remaining failure surface is therefore narrow and mechanical rather than diffuse, which is the most useful thing an error analysis can conclude. Schema and commonsense checks, by contrast, are near-saturated (78/80 queries), parse failures are 0, and no query is ever delivered empty.

Mechanism	ΔFPR	Δscore	verdict
Deterministic mechanisms			
Window-anchored relayout[a]	$+10.0$	$+5.3$	adopted
Pin timed windows[b]	$+6.67$	$+6.00$	adopted
Repair transport gaps[b]	$+3.33$	$+2.72$	adopted
Convergence stop[b]	± 0.0	$+2.28$	adopted
Language-model stages			
LLM parse audit[c]	± 0.0	-3.20	rejected
LLM plan refine[c]	± 0.0	-0.02	rejected
Extraction upper bound			
Oracle constraints[d]	-1.25	-1.30	no gain

Table 2: Ablations. [a] n=80, 90 s, solver-only and end-to-end agree. [b] n=30, 90 s. [c] n=30, 90 s, DeepInfra `Qwen3.6-27B`, shared all-LLM-off baseline; LLM activity verified per query. [d] 80-query split at 900 s: the solver fed oracle constraints instead of parsed ones. Screens are paired deltas at reduced budget; they rank mechanisms and do not estimate final performance.

5 Method: the model in the outer loop

The system contains no language model; building it was almost entirely model-driven. Over five months the repository accumulated 759 commits and about 69k lines of Python, of which 172 commits carry an AI co-author trailer, alongside 895 tests. Humans owned the architecture, the invariants, the measurement contract and which experiment to run next; the agent wrote most of the implementation and kept the documentation consistent with it. Three practices did the load-bearing work, and they transfer to any agent-built codebase:

Invariants as a checked-in contract. A single file states what may never change — acceptance before output, all facts from the environment, no oracle leakage, disjunctions kept branch-local, defaults make zero LLM calls, upstream evaluator reproduced exactly — and is loaded into every agent session. Agents are excellent at local optimization and will happily satisfy a metric by weakening a check; the invariants exist to make that unavailable rather than to be remembered.

Two tiers of evidence, and only one promotes. A fast paired screen on a subset triages a change on the developer's machine; only a whole-split run under a pinned contract, from a pinned revision, on a separate machine, can flip a default. Rows record data source, split, budget, flags, parallelism and evaluator revision, and rows straddling an evaluator sync are never compared. This is what let us reject our own plausible ideas: the preference-packing mechanisms, the extra-seed sweep and the LLM stages all sounded good and all failed their measurement.

Assert that a component ran before believing its number. Both LLM measurements were initially wrong, in opposite directions, for the same reason. A first parse-audit screen looked $+1.97$ — because at that timeout 10 of 30 audits silently expired and fell back to the deterministic parse, so the "benefit" was measuring the deterministic path with extra latency; raising the timeout until all 30 fired flipped the verdict to -3.20. Earlier, a ledger row labelled `llm_parse=true` recorded a configuration that had in fact received HTTP 404 for every call. A silently-degrading component is the default failure mode of LLM integration, and an A/B where the treatment never happened is indistinguishable from a null result unless the harness counts calls. We now require an activity counter in telemetry for every optional stage, and treat a green screen without it as unmeasured.

6 Conclusion

We submit a deterministic solver-first planner that scores 57.99 overall on the released `familiar_100` split, never fabricates an itinerary fact, and cannot ship a plan its checkers rejected. It contains no language model, in a language-agent challenge, on a host that offers one for free — because in the two roles we could measure, the model cost score and committed nothing, while the points sat in a single missing symbolic move worth $+10$ FPR. The generalization we would draw is not "LLMs do not help planning"; it is that in a benchmark whose difficulty is *grounding* under a strict verifier, the model's comparative advantage is in the loop that writes the solver, not the loop that runs it — and that telling those apart requires an evaluation harness willing to falsify your favourite idea, including your idea about the model.

Acknowledgements

We thank the ChinaTravel organizers for the benchmark, the sandbox environment and their prompt clarifications on the Phase 2 execution contract. The system was developed with substantial assistance from LLM-based coding agents under the review protocol described in Section 5.

References

[Kambhampati *et al.*, 2024] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can't plan, but can help planning in LLM-Modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[Perron and Didier, 2024] Laurent Perron and Frédéric Didier. CP-SAT. https://developers.google.com/optimization/cp/cp_solver, 2024. Google OR-Tools.

[Shao *et al.*, 2024] Jie-Jing Shao, Bo-Wen Zhang, Xiao-Wen Yang, Bai-Zhi Chen, Si-Yu Han, Jing-Hao Pang, Wen-Da Wei, Guohao Cai, Zhenhua Dong, Lan-Zhe Guo, and Yu-Feng Li. ChinaTravel: An open-ended benchmark for language agents in Chinese travel planning. *arXiv preprint arXiv:2412.13682*, 2024.

[Xie *et al.*, 2024] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. TravelPlanner: A benchmark for real-world planning with language agents. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.